TECHNICAL IMPLEMENTATION PLAN

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Project Code**: OG-AI-MON-2025

---

1. IMPLEMENTATION ROADMAP

1.1 Master Schedule

```
                                    18-MONTH IMPLEMENTATION ROADMAP

          M1   M2   M3   M4   M5   M6   M7   M8   M9   M10  M11  M12  M13  M14  M15  M16  M17  M18

PHASE 1                                                Discovery & Requirements
PHASE 2                                                Architecture & Design
PHASE 3                                      Development (6 Sprints)
PHASE 4                                               Pilot Deployment
PHASE 5                                                          Full Rollout
PHASE 6                                                                 Hypercare


MILESTONES:
  M2:  BRD Approved
  M4:  Architecture Signed Off
  M7:  Data Pipeline Complete
  M9:  ML Models Validated
  M11: Pilot Go-Live
  M14: Full Go-Live
  M16: Handover Complete
  M18: Warranty End
```

---

2. PHASE-WISE IMPLEMENTATION DETAILS

PHASE 1: DISCOVERY & REQUIREMENTS (Weeks 1-8)

Week 1-2: Stakeholder Engagement

Day	Activity	Participants	Output
D1	Kickoff meeting	All stakeholders	Project charter
D2-3	Ops team interviews	Plant managers, engineers	Pain points list
D4-5	IT/OT interviews	IT director, SCADA team	Integration requirements
D6-7	HSE interviews	HSE manager	Safety requirements
D8-10	Maintenance interviews	Reliability, planners	PM requirements

Week 3-4: Technical Discovery

Activity	Tasks	Deliverable
Data Audit	Inventory sensors, historians, data quality	Data audit report
System Inventory	Map existing systems (DCS, SCADA, ERP)	Integration map
Network Assessment	Bandwidth, latency, security zones	Network report
Infrastructure	Server, storage, edge locations	Infra requirements

Week 5-6: Requirements Definition

Requirement Type	Method	Deliverable
Functional	Workshops, use case modeling	Use case document
Non-Functional	Capacity planning, SLA definition	NFR document
Integration	API review, protocol analysis	Integration specs
Security	Threat modeling, compliance review	Security requirements

Week 7-8: BRD Finalization

Activity	Participants	Output
Draft BRD review	Core team	Reviewed BRD
Stakeholder review	All stakeholders	Comments addressed
Final approval	Steering committee	Signed BRD

Deliverables Checklist - Phase 1

- ☐ Stakeholder interview summaries
- ☐ Data audit report
- ☐ Integration requirements
- ☐ Use case document
- ☐ BRD (signed)
- ☐ Project charter
- ☐ Risk register v1

PHASE 2: ARCHITECTURE & DESIGN (Weeks 9-16)

Week 9-10: High-Level Architecture

Component	Design Activity	Output
Edge Layer	Edge node sizing, placement	Edge architecture
Platform Layer	Cloud/on-prem, K8s sizing	Platform architecture
Data Layer	Database selection, schema	Data architecture
ML Layer	Model serving, MLOps	ML architecture

Component	Design Activity	Output
Security	Zones, firewall rules	Security architecture

Week 11-12: Technology Selection

Category	Evaluation Criteria	Selected
Edge Compute	Performance, power, ATEX	Jetson AGX Orin
Container Platform	Enterprise support, security	OpenShift
ML Platform	On-prem, scalability	Kubeflow
Time-Series DB	Performance, retention	InfluxDB Enterprise
LLM	On-prem, fine-tuning	Llama 3.1 70B

Week 13-14: Detailed Design

Design Document	Author	Reviewer
Database schema	Data Engineer	DBA
API specifications	Backend Dev	Solution Architect
ML pipeline design	ML Engineer	Technical Lead
Network topology	Network Engineer	Security Engineer
Edge firmware design	Embedded Dev	Hardware Engineer

Week 15-16: Design Review & Approval

Review Type	Participants	Gate
Architecture review	Technical team	Must pass
Security review	Security team	Must pass
HSE review	HSE	Must pass
Steering approval	Executive sponsors	Must pass

Deliverables Checklist - Phase 2

- ☐ High-Level Design (HLD)
- ☐ Low-Level Design (LLD)
- ☐ Technology selection matrix
- ☐ Hardware BOM
- ☐ Security design
- ☐ Approved architecture

PHASE 3: DEVELOPMENT (Weeks 17-40)

Sprint Structure (2-week sprints)

```
DEVELOPMENT PHASE BREAKDOWN (24 weeks = 12 sprints)


Sprint 1-2 (W17-20):  INFRASTRUCTURE & DATA PIPELINE
  Kubernetes cluster setup
```

```
   Kafka cluster deployment
   InfluxDB deployment
   OPC-UA gateway setup
   Initial data ingestion

Sprint 3-4 (W21-24):  DATA PIPELINE COMPLETION
   SCADA/Historian integration
   Data validation pipeline
   Feature engineering pipeline
   Data lake setup
   Monitoring setup

Sprint 5-6 (W25-28):  ML - ANOMALY DETECTION
   Training data preparation
   Autoencoder training
   Threshold calibration
   Edge model optimization
   Inference pipeline

Sprint 7-8 (W29-32):  ML - RUL PREDICTION
   CNN-LSTM model training
   Uncertainty quantification
   Model validation
   Cloud deployment
   Integration testing

Sprint 9-10 (W33-36):  COMPUTER VISION
   PPE detection model
   Corrosion detection model
   Video pipeline (DeepStream)
   Edge deployment
   Camera integration

Sprint 11-12 (W37-40):  RAG & APPLICATIONS
   Document ingestion
   RAG pipeline
   Dashboard development
   Mobile app
   SAP integration
   End-to-end testing
```

Sprint 1-2: Infrastructure & Data Pipeline

Week 17-18: Infrastructure Setup

Task	Assignee	Duration	Dependency
K8s cluster provisioning	DevOps	3 days	Hardware delivered
Node configuration	DevOps	2 days	K8s cluster
Storage setup (Ceph)	DevOps	2 days	K8s cluster
Network configuration	Network Eng	2 days	Firewall rules
Kafka deployment	Data Eng	2 days	K8s cluster
InfluxDB deployment	Data Eng	1 day	Storage
PostgreSQL deployment	DBA	1 day	Storage
Milvus deployment	Data Eng	1 day	Storage

Task	Assignee	Duration	Dependency

Week 19-20: Data Ingestion

Task	Assignee	Duration	Dependency
OPC-UA connector setup	Embedded Dev	3 days	OPC gateway
MQTT broker deployment	Data Eng	1 day	K8s
NiFi flow development	Data Eng	3 days	Kafka, OPC
PI to InfluxDB sync	Data Eng	2 days	Historian access
Data validation rules	Data Eng	2 days	NiFi flows
Initial testing	QA	2 days	All above

Sprint 3-4: Data Pipeline Completion

Task	Assignee	Duration
Feature engineering service	Data Eng	5 days
Flink streaming jobs	Data Eng	4 days
Data quality monitoring	Data Eng	3 days
Alerting on data issues	Data Eng	2 days
S3/MinIO cold storage	DevOps	2 days
Data retention policies	DBA	1 day
End-to-end data flow test	QA	3 days

Sprint 5-6: Anomaly Detection

Task	Assignee	Duration
Training data extraction	ML Eng	3 days
Data preprocessing	ML Eng	2 days
Autoencoder training	ML Eng	5 days
Threshold optimization	ML Eng	2 days
ONNX export	ML Eng	1 day
TensorRT optimization	ML Eng	2 days
Edge deployment	Embedded Dev	3 days
Integration testing	QA	2 days

Sprint 7-8: RUL Prediction

Task	Assignee	Duration
Historical failure data prep	Data Eng	3 days
Feature sequence creation	ML Eng	2 days
CNN-LSTM training	ML Eng	5 days
Hyperparameter tuning	ML Eng	3 days
MC Dropout uncertainty	ML Eng	2 days
Model validation	ML Eng	2 days
Triton deployment	ML Eng	2 days
API integration	Backend Dev	2 days

Sprint 9-10: Computer Vision

Task	Assignee	Duration
PPE dataset annotation	CV Eng	5 days
YOLOv8 PPE training	CV Eng	3 days
Corrosion annotation	CV Eng	5 days
YOLOv8 corrosion training	CV Eng	3 days
DeepStream pipeline	CV Eng	4 days
Camera integration	Embedded Dev	2 days
Edge deployment	Embedded Dev	2 days
Validation testing	QA	3 days

Sprint 11-12: RAG & Applications

Task	Assignee	Duration
Document ingestion pipeline	RAG Eng	4 days
Embedding generation	RAG Eng	2 days
Milvus indexing	RAG Eng	2 days
LLM fine-tuning	RAG Eng	5 days
RAG API development	Backend Dev	3 days
Dashboard frontend	Frontend Dev	8 days
Mobile app	Mobile Dev	8 days
SAP PM integration	Backend Dev	4 days
End-to-end testing	QA	5 days

Deliverables Checklist - Phase 3

- ☐ Working data pipeline
- ☐ Anomaly detection model (edge)
- ☐ RUL prediction model (cloud)
- ☐ PPE detection model (edge)
- ☐ Corrosion detection model (edge)
- ☐ RAG knowledge assistant
- ☐ Control room dashboard
- ☐ Mobile app
- ☐ SAP PM integration
- ☐ Test reports

PHASE 4: PILOT DEPLOYMENT (Weeks 41-52)

Week 41-44: Installation & Configuration

Task	Location	Duration	Team
Edge hardware installation	Refinery A	5 days	HW Eng, Vendor
Camera installation	Refinery A	3 days	HW Eng, Vendor
Sensor commissioning	Refinery A	3 days	HW Eng
Network configuration	Refinery A	2 days	Network Eng
Edge software deployment	Remote	2 days	DevOps
Platform configuration	Remote	2 days	DevOps

Task	Location	Duration	Team
Integration testing	Refinery A	3 days	QA

Week 45-48: Shadow Mode Operation

Activity	Duration	Metrics
Data flow validation	2 weeks	Data completeness >99%
Model performance	2 weeks	Anomaly detection <5% FP
Alert validation	2 weeks	Manual verification of alerts
User feedback	Ongoing	Daily feedback sessions

Week 49-52: Live Mode & Refinement

Activity	Duration	Success Criteria
Live alert activation	2 weeks	Response time <15 min
Work order automation	1 week	SAP integration working
Model tuning	2 weeks	Metrics meet targets
User training	1 week	100% users trained
Documentation	1 week	Runbooks complete
Pilot review	1 week	Go/No-Go decision

Pilot Success Criteria

KPI	Target	Measurement
Failure predictions	>5 true positives	Maintenance records
False positive rate	<2%	Alert analysis
PPE compliance	>95%	CV detection logs
User satisfaction	>4.0/5.0	Survey
System uptime	>99%	Monitoring

PHASE 5: FULL ROLLOUT (Weeks 53-68)

Rollout Schedule

Month	Facility	Assets	Users	Team
M14	Refinery B	100	25	Team Alpha
M14	Refinery C	100	25	Team Beta
M15	Offshore A	75	20	Team Alpha
M15	Offshore B	75	20	Team Beta
M16	Pipeline Network	150	15	Team Alpha
M16	Gas Processing	125	20	Team Beta
Total	**7 sites**	**625**	**125**	

Site Rollout Checklist

```
PRE-DEPLOYMENT (Week -2)
  Site survey completed
  Network readiness confirmed
  Hardware shipped
  Site coordinator assigned
  Training scheduled

DEPLOYMENT (Week 1)
  Hardware installed
  Network connected
  Edge software deployed
  Cameras configured
  Sensors validated
  Integration tested

GO-LIVE (Week 2)
  Shadow mode (3 days)
  Live mode activation
  User training completed
  Support handover
  Documentation delivered
```

PHASE 6: HYPERCARE & HANDOVER (Weeks 69-72)

Hypercare Support

Support Level	Response Time	Coverage
P1 (Critical)	15 minutes	24/7
P2 (High)	1 hour	24/7
P3 (Medium)	4 hours	Business hours
P4 (Low)	24 hours	Business hours

Knowledge Transfer

Session	Duration	Audience	Topics
Platform Admin	2 days	IT Ops	K8s, monitoring, backup
ML Operations	2 days	Data Science	MLflow, retraining, drift
Edge Support	1 day	Field Eng	Troubleshooting, updates
Application Support	1 day	IT Support	Dashboard, mobile, RAG

Handover Documents

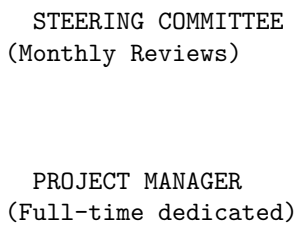
- ☐ Operations manual
- ☐ Troubleshooting guide
- ☐ Runbook for common issues
- ☐ Architecture documentation
- ☐ Source code repository
- ☐ Model training pipelines
- ☐ Vendor contact list

3. TEAM ALLOCATION

3.1 Resource Matrix

Role	Phase 1	Phase 2	Phase 3	Phase 4	Phase 5	Phase 6
Project Manager	1.0	1.0	1.0	1.0	1.0	0.5
Solution Architect	0.5	1.0	0.5	0.25	0.25	0.25
Hardware Engineer	0.25	1.0	0.5	1.0	1.0	0.25
Data Engineer	0.5	0.5	2.0	1.0	0.5	0.25
ML Engineer	-	0.25	2.0	1.0	0.5	0.25
CV Engineer	-	0.25	1.0	0.5	0.25	0.1
RAG/NLP Engineer	-	0.25	1.0	0.5	0.25	0.1
Backend Developer	-	0.5	2.0	1.0	0.5	0.25
Frontend Developer	-	0.25	1.0	0.5	0.25	0.1
Mobile Developer	-	-	1.0	0.25	0.1	-
DevOps Engineer	-	0.5	1.0	1.0	1.0	0.5
QA Engineer	0.25	0.5	2.0	1.0	1.0	0.5
Security Engineer	0.25	0.5	0.25	0.25	0.25	0.1
Total FTEs	**2.75**	**6.5**	**14.25**	**9.0**	**6.85**	**3.15**

3.2 Team Structure



PLATFORM TEAM	ML TEAM	APPLICATION TEAM
Solution Architect	ML Engineer (Lead)	Backend Dev (Lead)
DevOps Engineer	ML Engineer	Backend Developer
Data Engineer (2)	CV Engineer	Frontend Developer
Hardware Engineer	RAG/NLP Engineer	Mobile Developer
Network Engineer		

QUALITY TEAM
QA Engineer (Lead)
QA Engineer
Security Engineer

---

4. TASK-BY-TASK IMPLEMENTATION GUIDE

4.1 Data Pipeline Implementation

Task 1: Kafka Cluster Setup

Objective: Deploy production Kafka cluster for event streaming

Steps:

```
# Step 1: Create Kafka namespace
kubectl create namespace kafka

# Step 2: Deploy Strimzi operator
helm repo add strimzi https://strimzi.io/charts/
helm install strimzi-kafka-operator strimzi/strimzi-kafka-operator -n kafka

# Step 3: Create Kafka cluster (3 brokers, 3 ZK)
kubectl apply -f - <<EOF
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata:
  name: oilgas-kafka
  namespace: kafka
spec:
  kafka:
    version: 3.6.0
    replicas: 3
    listeners:
      - name: plain
        port: 9092
        type: internal
```

```
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
    config:
      offsets.topic.replication.factor: 3
      transaction.state.log.replication.factor: 3
      min.insync.replicas: 2
    storage:
      type: persistent-claim
      size: 500Gi
      class: ceph-ssd
  zookeeper:
    replicas: 3
    storage:
      type: persistent-claim
      size: 100Gi
      class: ceph-ssd
EOF

# Step 4: Create topics
kubectl apply -f - <<EOF
apiVersion: kafka.strimzi.io/v1beta2
kind: KafkaTopic
metadata:
  name: sensor-readings
  namespace: kafka
  labels:
    strimzi.io/cluster: oilgas-kafka
spec:
  partitions: 20
  replicas: 3
  config:
    retention.ms: 604800000  # 7 days
    segment.bytes: 1073741824
EOF

# Step 5: Verify
kubectl get kafka -n kafka
kubectl get kafkatopic -n kafka
```

Validation: - [ ] 3 Kafka brokers running - [ ] All topics created - [ ] Producer/consumer test successful - [ ] Monitoring dashboard accessible

---

Task 2: OPC-UA Integration

Objective: Connect to plant OPC-UA servers and stream data to Kafka

Configuration (`nifi/opc-ua-flow.xml`):

```
<flow>
  <processor name="ConsumeOPCUA">
    <property name="OPC UA Server URL">opc.tcp://scada-server:4840</property>
```

```
    <property name="Security Mode">SignAndEncrypt</property>
    <property name="Security Policy">Basic256Sha256</property>
    <property name="Node ID Pattern">ns=2;s=Sensors.*</property>
    <property name="Subscription Interval">1000</property>
  </processor>

  <processor name="TransformToJSON">
    <property name="Schema">
      {
        "sensor_id": "${opcua.nodeId}",
        "timestamp": "${opcua.serverTimestamp}",
        "value": ${opcua.value},
        "quality": ${opcua.statusCode}
      }
    </property>
  </processor>

  <processor name="PublishKafka">
    <property name="Kafka Brokers">kafka-bootstrap:9092</property>
    <property name="Topic">sensor-readings</property>
    <property name="Partition Key">${sensor_id}</property>
  </processor>
</flow>
```

Validation: - [ ] Connection to OPC-UA server established - [ ] Sensor values flowing to Kafka - [ ] Message rate matches expected (~1000/sec) - [ ] No data loss during 24-hour test

---

Task 3: InfluxDB Setup

Objective: Deploy InfluxDB for time-series storage

Steps:

```
# Step 1: Deploy InfluxDB
helm install influxdb influxdata/influxdb2 \
  --namespace observability \
  --set persistence.enabled=true \
  --set persistence.size=2Ti \
  --set persistence.storageClass=ceph-ssd \
  --set resources.requests.memory=32Gi \
  --set resources.limits.memory=64Gi

# Step 2: Configure bucket and retention
influx bucket create \
  --name sensor_data \
  --retention 7d \
  --org oilgas

influx bucket create \
  --name sensor_data_downsampled \
  --retention 365d \
  --org oilgas

# Step 3: Create continuous query for downsampling
```

```
influx task create <<EOF
option task = {name: "downsample_1m", every: 1m}

from(bucket: "sensor_data")
  |> range(start: -1m)
  |> aggregateWindow(every: 1m, fn: mean, createEmpty: false)
  |> to(bucket: "sensor_data_downsampled")
EOF
```

4.2 ML Model Implementation

Task 4: Anomaly Detection Training

Training Script (`ml/train_anomaly.py`):

```python
#!/usr/bin/env python3
"""
Anomaly Detection Model Training

Usage: python train_anomaly.py --config config.yaml
"""

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, Dataset
import numpy as np
from pathlib import Path
import mlflow
from typing import Tuple
import yaml
import argparse

class SensorDataset(Dataset):
    """Dataset for sensor features"""
    def __init__(self, features: np.ndarray):
        self.features = torch.FloatTensor(features)

    def __len__(self):
        return len(self.features)

    def __getitem__(self, idx):
        return self.features[idx]

class Autoencoder(nn.Module):
    """Autoencoder for anomaly detection"""
    def __init__(self, input_dim: int = 15, latent_dim: int = 3):
        super().__init__()

        self.encoder = nn.Sequential(
            nn.Linear(input_dim, 10),
            nn.ReLU(),
            nn.BatchNorm1d(10),
            nn.Dropout(0.2),
            nn.Linear(10, 5),
```

```
            nn.ReLU(),
            nn.BatchNorm1d(5),
            nn.Linear(5, latent_dim)
        )

        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 5),
            nn.ReLU(),
            nn.BatchNorm1d(5),
            nn.Linear(5, 10),
            nn.ReLU(),
            nn.BatchNorm1d(10),
            nn.Dropout(0.2),
            nn.Linear(10, input_dim)
        )

    def forward(self, x):
        latent = self.encoder(x)
        reconstructed = self.decoder(latent)
        return reconstructed, latent

def load_training_data(data_path: str) -> Tuple[np.ndarray, np.ndarray]:
    """Load and split training data"""
    # Load from InfluxDB or file
    data = np.load(data_path)

    # Split 80/20
    split_idx = int(len(data) * 0.8)
    train_data = data[:split_idx]
    val_data = data[split_idx:]

    return train_data, val_data

def normalize_data(train: np.ndarray, val: np.ndarray) -> Tuple[np.ndarray, np.ndarray, dict]:
    """Z-score normalization"""
    mean = train.mean(axis=0)
    std = train.std(axis=0) + 1e-8

    train_norm = (train - mean) / std
    val_norm = (val - mean) / std

    stats = {'mean': mean.tolist(), 'std': std.tolist()}
    return train_norm, val_norm, stats

def train_model(config: dict):
    """Main training function"""

    # MLflow tracking
    mlflow.set_tracking_uri(config['mlflow_uri'])
    mlflow.set_experiment("anomaly_detection")

    with mlflow.start_run():
        # Log parameters
        mlflow.log_params(config['model'])
```

```
# Load data
train_data, val_data = load_training_data(config['data_path'])
train_norm, val_norm, stats = normalize_data(train_data, val_data)

# Create datasets
train_dataset = SensorDataset(train_norm)
val_dataset = SensorDataset(val_norm)

train_loader = DataLoader(
    train_dataset,
    batch_size=config['training']['batch_size'],
    shuffle=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=config['training']['batch_size']
)

# Initialize model
model = Autoencoder(
    input_dim=config['model']['input_dim'],
    latent_dim=config['model']['latent_dim']
)

criterion = nn.MSELoss()
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=config['training']['learning_rate']
)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, patience=5, factor=0.5
)

# Training loop
best_val_loss = float('inf')
patience_counter = 0

for epoch in range(config['training']['epochs']):
    # Train
    model.train()
    train_loss = 0
    for batch in train_loader:
        optimizer.zero_grad()
        reconstructed, _ = model(batch)
        loss = criterion(reconstructed, batch)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()

    train_loss /= len(train_loader)

    # Validate
    model.eval()
```

```
        val_loss = 0
        reconstruction_errors = []
        with torch.no_grad():
            for batch in val_loader:
                reconstructed, _ = model(batch)
                loss = criterion(reconstructed, batch)
                val_loss += loss.item()

                # Per-sample reconstruction error
                errors = torch.mean((batch - reconstructed) ** 2, dim=1)
                reconstruction_errors.extend(errors.numpy())

        val_loss /= len(val_loader)
        scheduler.step(val_loss)

        # Log metrics
        mlflow.log_metrics({
            'train_loss': train_loss,
            'val_loss': val_loss,
            'lr': optimizer.param_groups[0]['lr']
        }, step=epoch)

        # Early stopping
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            patience_counter = 0

            # Save best model
            torch.save(model.state_dict(), 'best_model.pt')
        else:
            patience_counter += 1
            if patience_counter >= config['training']['patience']:
                print(f"Early stopping at epoch {epoch}")
                break

        print(f"Epoch {epoch}: train_loss={train_loss:.6f}, val_loss={val_loss:.6f}")

    # Calculate threshold (mean + 2*std of reconstruction errors)
    errors = np.array(reconstruction_errors)
    threshold_mean = float(np.mean(errors))
    threshold_std = float(np.std(errors))

    # Save model and artifacts
    mlflow.log_artifact('best_model.pt')
    mlflow.log_dict({
        'normalization': stats,
        'threshold_mean': threshold_mean,
        'threshold_std': threshold_std
    }, 'config.json')

    # Export to ONNX
    model.load_state_dict(torch.load('best_model.pt'))
    model.eval()
    dummy_input = torch.randn(1, config['model']['input_dim'])
```

```
        torch.onnx.export(
            model,
            dummy_input,
            'model.onnx',
            input_names=['input'],
            output_names=['reconstructed', 'latent'],
            dynamic_axes={'input': {0: 'batch'}}
        )
        mlflow.log_artifact('model.onnx')

        print(f"Training complete. Threshold: {threshold_mean:.6f} ± {threshold_std:.6f}")

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--config', required=True)
    args = parser.parse_args()

    with open(args.config) as f:
        config = yaml.safe_load(f)

    train_model(config)
```

Validation: - [ ] Model converges (val_loss decreasing) - [ ] Reconstruction error on normal data < threshold - [ ] ONNX export successful - [ ] Model registered in MLflow

---

Task 5: Edge Deployment

Deployment Script (`edge/deploy_model.py`):

```
#!/usr/bin/env python3
"""
Deploy model to edge nodes

Usage: python deploy_model.py --model-uri s3://models/anomaly/v1.0 --nodes edge-001,edge-002
"""

import tensorrt as trt
import pycuda.driver as cuda
import pycuda.autoinit
import numpy as np
from pathlib import Path
import argparse
import paramiko
import json

def convert_onnx_to_tensorrt(onnx_path: str, output_path: str, precision: str = 'fp16'):
    """Convert ONNX model to TensorRT engine"""
    logger = trt.Logger(trt.Logger.WARNING)
    builder = trt.Builder(logger)
    network = builder.create_network(1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH))
    parser = trt.OnnxParser(network, logger)

    with open(onnx_path, 'rb') as f:
        if not parser.parse(f.read()):
```

```
            for error in range(parser.num_errors):
                print(parser.get_error(error))
            raise RuntimeError("ONNX parsing failed")

    config = builder.create_builder_config()
    config.set_memory_pool_limit(trt.MemoryPoolType.WORKSPACE, 1 << 30)  # 1GB

    if precision == 'fp16':
        config.set_flag(trt.BuilderFlag.FP16)

    profile = builder.create_optimization_profile()
    profile.set_shape('input', (1, 15), (16, 15), (32, 15))  # min, opt, max
    config.add_optimization_profile(profile)

    engine = builder.build_serialized_network(network, config)

    with open(output_path, 'wb') as f:
        f.write(engine)

    print(f"TensorRT engine saved to {output_path}")
    return output_path

def deploy_to_node(node_ip: str, engine_path: str, config_path: str, ssh_key: str):
    """Deploy model to edge node via SSH"""
    ssh = paramiko.SSHClient()
    ssh.set_missing_host_key_policy(paramiko.AutoAddPolicy())
    ssh.connect(node_ip, username='oilgas', key_filename=ssh_key)

    sftp = ssh.open_sftp()

    # Upload files
    remote_model_dir = '/opt/models/anomaly'
    sftp.put(engine_path, f'{remote_model_dir}/model.engine')
    sftp.put(config_path, f'{remote_model_dir}/config.json')

    # Restart inference service
    stdin, stdout, stderr = ssh.exec_command('systemctl restart inference-service')
    exit_code = stdout.channel.recv_exit_status()

    if exit_code != 0:
        print(f"Error restarting service on {node_ip}: {stderr.read().decode()}")
        return False

    # Verify deployment
    stdin, stdout, stderr = ssh.exec_command('curl -s http://localhost:8080/health')
    health = json.loads(stdout.read().decode())

    ssh.close()

    if health.get('status') == 'healthy':
        print(f"Model deployed successfully to {node_ip}")
        return True
    else:
        print(f"Deployment verification failed on {node_ip}")
```

```
        return False

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--model-uri', required=True)
    parser.add_argument('--nodes', required=True, help='Comma-separated node IPs')
    parser.add_argument('--precision', default='fp16')
    parser.add_argument('--ssh-key', default='~/.ssh/oilgas_edge')
    args = parser.parse_args()

    # Download model from S3
    # ... (implementation)

    # Convert to TensorRT
    engine_path = convert_onnx_to_tensorrt(
        'model.onnx',
        'model.engine',
        args.precision
    )

    # Deploy to each node
    nodes = args.nodes.split(',')
    results = {}
    for node in nodes:
        success = deploy_to_node(
            node.strip(),
            engine_path,
            'config.json',
            args.ssh_key
        )
        results[node] = 'success' if success else 'failed'

    print("\nDeployment Summary:")
    for node, status in results.items():
        print(f"  {node}: {status}")

if __name__ == '__main__':
    main()
```

4.3 RAG Implementation

Task 6: Document Ingestion

Ingestion Script (`rag/ingest_documents.py`):

```
#!/usr/bin/env python3
"""
Document ingestion pipeline for RAG

Usage: python ingest_documents.py --source /data/documents --batch-size 100
"""

import os
from pathlib import Path
```

```python
from typing import List, Dict, Generator
import hashlib
from datetime import datetime
import json

from unstructured.partition.auto import partition
from langchain.text_splitter import RecursiveCharacterTextSplitter
from sentence_transformers import SentenceTransformer
from pymilvus import connections, Collection, CollectionSchema, FieldSchema, DataType
import psycopg2
from psycopg2.extras import execute_values

class DocumentIngester:
    def __init__(self, config: dict):
        self.config = config

        # Initialize embedding model
        self.embedder = SentenceTransformer(config['embedding_model'])

        # Initialize text splitter
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=config['chunk_size'],
            chunk_overlap=config['chunk_overlap'],
            separators=["\n\n", "\n", ". ", " ", ""]
        )

        # Connect to Milvus
        connections.connect(
            host=config['milvus_host'],
            port=config['milvus_port']
        )
        self._init_milvus_collection()

        # Connect to PostgreSQL
        self.pg_conn = psycopg2.connect(config['postgres_dsn'])

    def _init_milvus_collection(self):
        """Create Milvus collection if not exists"""
        collection_name = self.config['collection_name']

        if collection_name in [c.name for c in connections.list_collections()]:
            self.collection = Collection(collection_name)
            return

        fields = [
            FieldSchema(name="chunk_id", dtype=DataType.VARCHAR, max_length=36, is_primary=True),
            FieldSchema(name="document_id", dtype=DataType.VARCHAR, max_length=36),
            FieldSchema(name="document_type", dtype=DataType.VARCHAR, max_length=50),
            FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=self.config['embedding_dim']),
            FieldSchema(name="content_preview", dtype=DataType.VARCHAR, max_length=500)
        ]
        schema = CollectionSchema(fields, description="Document embeddings")

        self.collection = Collection(collection_name, schema)
```

```
        # Create index
        index_params = {
            "metric_type": "COSINE",
            "index_type": "IVF_PQ",
            "params": {"nlist": 1024, "m": 16, "nbits": 8}
        }
        self.collection.create_index("embedding", index_params)

    def parse_document(self, file_path: Path) -> Dict:
        """Parse document using unstructured"""
        elements = partition(str(file_path))

        # Extract text content
        text_content = "\n\n".join([str(el) for el in elements])

        # Extract metadata
        metadata = {
            'file_name': file_path.name,
            'file_type': file_path.suffix[1:],
            'file_size': file_path.stat().st_size,
            'checksum': self._compute_checksum(file_path)
        }

        return {
            'content': text_content,
            'metadata': metadata
        }

    def _compute_checksum(self, file_path: Path) -> str:
        """Compute SHA256 checksum"""
        sha256_hash = hashlib.sha256()
        with open(file_path, 'rb') as f:
            for byte_block in iter(lambda: f.read(4096), b""):
                sha256_hash.update(byte_block)
        return sha256_hash.hexdigest()

    def chunk_document(self, content: str) -> List[str]:
        """Split document into chunks"""
        return self.splitter.split_text(content)

    def embed_chunks(self, chunks: List[str]) -> List[List[float]]:
        """Generate embeddings for chunks"""
        return self.embedder.encode(chunks, normalize_embeddings=True).tolist()

    def store_document_metadata(self, doc_id: str, metadata: Dict, chunk_count: int):
        """Store document metadata in PostgreSQL"""
        with self.pg_conn.cursor() as cur:
            cur.execute("""
                INSERT INTO knowledge.documents
                (document_id, file_name, file_path, file_type, file_size_bytes,
                 checksum_sha256, indexed_at, status)
                VALUES (%s, %s, %s, %s, %s, %s, %s, 'indexed')
                ON CONFLICT (document_id) DO UPDATE SET
```

```
                indexed_at = EXCLUDED.indexed_at,
                status = 'indexed'
        """, (
            doc_id,
            metadata['file_name'],
            metadata.get('file_path'),
            metadata['file_type'],
            metadata['file_size'],
            metadata['checksum'],
            datetime.utcnow()
        ))
    self.pg_conn.commit()

def store_chunks(self, doc_id: str, doc_type: str, chunks: List[str], embeddings: List[List[float]]):
    """Store chunks in PostgreSQL and Milvus"""
    chunk_data = []
    milvus_data = []

    for i, (chunk, embedding) in enumerate(zip(chunks, embeddings)):
        chunk_id = f"{doc_id}_{i:04d}"

        # PostgreSQL data
        chunk_data.append((
            chunk_id,
            doc_id,
            i,
            chunk,
            len(chunk.split()),
            self.config['embedding_model'],
            chunk_id  # embedding_id matches chunk_id
        ))

        # Milvus data
        milvus_data.append({
            'chunk_id': chunk_id,
            'document_id': doc_id,
            'document_type': doc_type,
            'embedding': embedding,
            'content_preview': chunk[:500]
        })

    # Insert into PostgreSQL
    with self.pg_conn.cursor() as cur:
        execute_values(cur, """
            INSERT INTO knowledge.chunks
            (chunk_id, document_id, chunk_index, content, token_count,
             embedding_model, embedding_id)
            VALUES %s
            ON CONFLICT (chunk_id) DO NOTHING
        """, chunk_data)
    self.pg_conn.commit()

    # Insert into Milvus
    self.collection.insert([
```

```
            [d['chunk_id'] for d in milvus_data],
            [d['document_id'] for d in milvus_data],
            [d['document_type'] for d in milvus_data],
            [d['embedding'] for d in milvus_data],
            [d['content_preview'] for d in milvus_data]
        ])
        self.collection.flush()

    def ingest_document(self, file_path: Path, doc_type: str = 'manual') -> Dict:
        """Full ingestion pipeline for single document"""
        # Generate document ID
        doc_id = hashlib.md5(str(file_path).encode()).hexdigest()[:36]

        # Parse
        parsed = self.parse_document(file_path)

        # Chunk
        chunks = self.chunk_document(parsed['content'])

        # Embed
        embeddings = self.embed_chunks(chunks)

        # Store
        self.store_document_metadata(doc_id, parsed['metadata'], len(chunks))
        self.store_chunks(doc_id, doc_type, chunks, embeddings)

        return {
            'document_id': doc_id,
            'chunk_count': len(chunks),
            'file_name': parsed['metadata']['file_name']
        }

    def ingest_directory(self, directory: Path, batch_size: int = 100) -> Generator:
        """Ingest all documents in directory"""
        supported_extensions = {'.pdf', '.docx', '.doc', '.txt', '.html'}

        files = [
            f for f in directory.rglob('*')
            if f.is_file() and f.suffix.lower() in supported_extensions
        ]

        for i, file_path in enumerate(files):
            try:
                result = self.ingest_document(file_path)
                result['status'] = 'success'
                yield result

                if (i + 1) % batch_size == 0:
                    print(f"Processed {i + 1}/{len(files)} documents")

            except Exception as e:
                yield {
                    'file_name': file_path.name,
                    'status': 'error',
```

```
                    'error': str(e)
                }

def main():
    import argparse

    parser = argparse.ArgumentParser()
    parser.add_argument('--source', required=True)
    parser.add_argument('--batch-size', type=int, default=100)
    parser.add_argument('--config', default='config.yaml')
    args = parser.parse_args()

    import yaml
    with open(args.config) as f:
        config = yaml.safe_load(f)

    ingester = DocumentIngester(config['rag'])

    source_path = Path(args.source)

    success_count = 0
    error_count = 0

    for result in ingester.ingest_directory(source_path, args.batch_size):
        if result['status'] == 'success':
            success_count += 1
        else:
            error_count += 1
            print(f"Error processing {result['file_name']}: {result['error']}")

    print(f"\nIngestion complete: {success_count} success, {error_count} errors")

if __name__ == '__main__':
    main()
```

---

5. TESTING STRATEGY

5.1 Test Categories

Category	Scope	Tools	Frequency
Unit	Individual functions	Pytest	Every commit
Integration	Component interactions	Pytest + Docker	Every PR
System	End-to-end flows	Selenium, Postman	Daily
Performance	Load, stress, soak	JMeter, Locust	Weekly
Security	Vulnerability, penetration	OWASP ZAP, Nessus	Sprint end
UAT	User acceptance	Manual + scripts	Phase end

5.2 Test Cases Summary

Data Pipeline Tests

- ☐ TP-001: OPC-UA connection and data flow
- ☐ TP-002: Kafka message throughput (10K/sec)
- ☐ TP-003: InfluxDB write performance (500K/sec)
- ☐ TP-004: Data validation and quality checks
- ☐ TP-005: End-to-end latency (<30 seconds)

ML Model Tests

- ☐ TM-001: Anomaly detection accuracy (>95% on test set)
- ☐ TM-002: RUL prediction RMSE (<15 days)
- ☐ TM-003: PPE detection mAP (>0.9)
- ☐ TM-004: Edge inference latency (<100ms)
- ☐ TM-005: Model drift detection

RAG Tests

- ☐ TR-001: Document ingestion (>100 docs/hour)
- ☐ TR-002: Query response time (<5 seconds)
- ☐ TR-003: Answer relevance (>90% user satisfaction)
- ☐ TR-004: Context retrieval accuracy (MRR >0.8)

API Tests

- ☐ TA-001: Endpoint availability (100%)
- ☐ TA-002: Response time P95 (<500ms)
- ☐ TA-003: Error handling and codes
- ☐ TA-004: Authentication/Authorization

Security Tests

- ☐ TS-001: Authentication bypass attempts
- ☐ TS-002: SQL injection
- ☐ TS-003: API rate limiting
- ☐ TS-004: Data encryption verification

---

6. DEPLOYMENT CHECKLIST

6.1 Pre-Deployment

```
PRE-DEPLOYMENT CHECKLIST


INFRASTRUCTURE
  Kubernetes cluster healthy (all nodes Ready)
  Storage available (>80% free)
  Network connectivity verified
  DNS configured
  SSL certificates valid

SECURITY
```

```
  Secrets rotated
  RBAC policies applied
  Network policies in place
  Security scan passed

APPLICATIONS
  All images built and pushed
  Helm charts validated
  ConfigMaps created
  Database migrations ready

DATA
  Database backups taken
  Test data loaded (staging)
  Feature store populated

MONITORING
  Prometheus scraping targets
  Grafana dashboards imported
  AlertManager rules configured
  Log aggregation working

ROLLBACK
  Previous deployment tagged
  Rollback procedure documented
  Database rollback script ready
```

6.2 Deployment Commands

```
# Step 1: Create namespace
kubectl create namespace oilgas-ai

# Step 2: Apply secrets
kubectl apply -f secrets.yaml -n oilgas-ai

# Step 3: Deploy databases
helm upgrade --install postgres bitnami/postgresql -f postgres-values.yaml -n oilgas-ai
helm upgrade --install influxdb influxdata/influxdb2 -f influxdb-values.yaml -n oilgas-ai
helm upgrade --install milvus milvus/milvus -f milvus-values.yaml -n oilgas-ai
helm upgrade --install redis bitnami/redis -f redis-values.yaml -n oilgas-ai

# Step 4: Deploy platform services
helm upgrade --install data-pipeline ./charts/data-pipeline -f values-prod.yaml -n oilgas-ai
helm upgrade --install ml-platform ./charts/ml-platform -f values-prod.yaml -n oilgas-ai
helm upgrade --install rag-service ./charts/rag-service -f values-prod.yaml -n oilgas-ai

# Step 5: Deploy applications
helm upgrade --install api-gateway ./charts/api-gateway -f values-prod.yaml -n oilgas-ai
helm upgrade --install dashboard ./charts/dashboard -f values-prod.yaml -n oilgas-ai

# Step 6: Verify
kubectl get pods -n oilgas-ai
kubectl get svc -n oilgas-ai
```

```
# Step 7: Smoke test
curl -s https://api.oilgas-ai.internal/health | jq
```

6.3 Post-Deployment Verification

```
POST-DEPLOYMENT VERIFICATION


HEALTH CHECKS
  All pods Running
  All services accessible
  Database connections verified
  API health endpoints responding

FUNCTIONAL VERIFICATION
  Login successful
  Dashboard loading
  Sensor data flowing
  Alerts triggering
  RAG queries working

MONITORING VERIFICATION
  Metrics appearing in Grafana
  Logs appearing in Kibana
  Traces appearing in Jaeger
  Alerts firing correctly

PERFORMANCE VERIFICATION
  API response time <500ms
  Dashboard load time <3s
  No error spikes in logs
```

---

7. SUMMARY

Documents Delivered

Document	File	Size
Blog Series Master Plan	`blog-series-master-plan.md`	50KB
Business Requirements (BRD)	`oil-gas-ai-monitoring-brd.md`	85KB
High-Level Design (HLD)	`oil-gas-ai-monitoring-hld.md`	45KB
Low-Level Design (LLD)	`oil-gas-ai-monitoring-lld.md`	60KB
Technical Implementation Plan	`oil-gas-ai-monitoring-technical-plan.md`	55KB

Key Metrics

Metric	Value
Total Project Duration	18 months
Team Size (Peak)	14.25 FTEs
Total Budget	$5.98M

Metric	Value
Hardware Cost	$1.52M
Sites Covered	7 facilities
Assets Monitored	1,000
Users Trained	170
Blog Posts Planned	158

END OF TECHNICAL IMPLEMENTATION PLAN